

ADAPTIVE CONTINUOUS AUTHENTICATION (ACA)

Consolidated Engineering Work Journals — Weeks 1 to 4

Course: UCS503P Software Engineering Project | Supervisor: Prof. Asif (Dr. Jeelani Asif) | Institution: Thapar Institute of Engineering & Technology, Patiala

Project Scope: Behavioral Biometrics & Keystroke Dynamics | Repository: github.com/Anjalikumari990/ucs503_project | Milestone: Prototype Verification (September 2026)

Student Name	Roll No.	Primary Engineering Domain & Core Module Ownership
Yatharth Kansal	1024030456	Machine Learning Pipeline, Biometric Feature Extraction & Trust Scoring Engine Lead
Anjali Kumari	1024030457	Frontend Dashboard, Flask REST API Gateway & Real-Time Client Event Streaming Lead
Ridhi Batra	1024030464	Automated Testing Suite, Privacy Architecture Auditing & Formal IEEE/UML Documentation Lead

Yatharth Kansal • Roll Number: 1024030456

Engineering Focus: Machine Learning Pipeline, Biometric Feature Extraction & Trust Scoring Engine Lead

Week 1: High-Precision Monotonic Event Logging & Privacy-Preserving Collection

- Engineered the low-level event interception collector (`code/collectors/keyboard_logger.py`) using `pynput` with monotonic clocking (`time.perf_counter`).
- Implemented client-side HMAC-SHA256 pseudonymization with an ephemeral 32-byte secret, discarding raw characters immediately to eliminate keylogger liabilities.
- Recorded baseline sessions (U01/S01, U01/S02) and verified zero negative time deltas alongside 100% absence of plaintext keystrokes.

Week 2: Keystroke Biometric Mathematics & 10-Second Sliding Window Feature Engineering

- Formulated the mathematical keystroke biometric metrics: dwell/hold time ($t_{\text{release}} - t_{\text{press}}$) and character flight time with n-key rollover support.
- Engineered 10-second temporal sliding windows (`code/src/features/keystroke_features.py`) extracting 5 core features: typing rate, mean hold, hold jitter, mean flight, and pause count.
- Enforced an active typing threshold (`is_active = character_count >= 5`) to prevent idle pauses from polluting statistical biometric distributions.

Week 3: Scikit-Learn Machine Learning Pipeline & One-Class Isolation Forest Model

- Formulated continuous authentication as an unsupervised one-class anomaly detection task, eliminating dependency on non-existent future adversary training data.
- Constructed an encapsulated Scikit-Learn Pipeline integrating median missing-value imputation, standard feature scaling, and an Isolation Forest (100 trees).
- Serialized the baseline model to `code/models/anomaly_model.pkl` and built an inference wrapper (`code/src/models/predict.py`) executing predictions in under 0.8 ms.

Week 4: Continuous Trust-Scoring Engine, Anomaly Mapping & Multi-Tempo Diagnosis

- Derived a clipped linear score mapping translating raw decision function outputs $[-0.10, +0.10]$ into an intuitive, normalized 0–100% continuous trust score.
- Implemented the 3-tier security policy engine (`code/src/authentication/decision.py`): TRUSTED ($\geq 70\%$, ALLOW), VERIFY (40–69%, PROMPT_REAUTH), and REJECT ($< 40\%$, LOCK_SESSION).
- Diagnosed empirical scoring variations between slow-tempo training data and fluent live typing, mathematically validating model behavior and defining multi-tempo calibration steps.

Core Deliverables & Verified Artifacts:

`code/collectors/keyboard_logger.py`, `code/src/features/keystroke_features.py`, `code/src/models/train_model.py`, `code/models/anomaly_model.pkl`, `code/src/trust_score.py`, `code/src/authentication/decision.py`, `code/src/engine.py`.

MACHINE LEARNING PIPELINE & BEHAVIORAL BIOMETRICS ARCHITECTURE

Mathematical Formulation: Dwell Time $T_{\text{hold}} = t_{\text{release}} - t_{\text{press}}$ • Flight Time $T_{\text{flight}} = t_{\text{press}}^{(k_{n+1})} - t_{\text{release}}^{(k_n)}$ • Active Gate: $N_{\text{chars}} \geq 5$
Feature Vector: `[chars_per_sec, mean_hold_ms, std_hold_jitter, mean_flight_ms, pause_count]` • Median Imputation • Standard Scaling
Anomaly Model & Policy: One-Class Isolation Forest (100 trees) • Linear Trust Metric (0–100%) • ALLOW / PROMPT_REAUTH / LOCK_SESSION

Anjali Kumari • Roll Number: 1024030457

Engineering Focus: Frontend Dashboard, Flask REST API Gateway & Real-Time Client Event Streaming Lead

Week 1: Flask Application Architecture, Session Security & State Isolation

- Architected the multi-session Flask web application (code/main.py) with an in-memory session cache bounded by a thread-safe threading.RLock().
- Configured strict enterprise security controls: HTTP-only cookies, SameSite restrictions, request payload caps (512 KB), and custom X-CSRF-Token validation.
- Built authenticated REST API routes (/api/start, /api/state, /api/window), ensuring unauthenticated calls are blocked with HTTP 401.

Week 2: Backend Integration Adapter & Inactive Window Policy Enforcement

- Constructed code/engine_adapter.py to seamlessly ingest variable-length client JSON event batches and map them into ML-ready pandas DataFrames.
- Implemented the Inactive Window Policy: if a window contains <5 characters, the system preserves the previous trust score and status without penalty, setting decision to HOLD.
- Added LRU caching (@lru_cache) for model loading, eliminating redundant disk I/O and maintaining sub-millisecond evaluation response times.

Week 3: In-Browser Client Keystroke Capture Engine & Web Crypto HMAC Hashing

- Engineered the client-side keystroke interception engine in code/static/dashboard.js using the W3C Web Crypto API (crypto.subtle) for in-browser HMAC-SHA256 signing.
- Enforced strict zero-plaintext transmission across the network: only 64-character hexadecimal hashes and relative microsecond timing deltas are transmitted to the backend.
- Integrated the Web Locks API (navigator.locks) for single-tab session ownership and bound visibilitychange listeners to pause capture when switching tabs.

Week 4: Dark-Mode SOC Web Dashboard & Dynamic SVG Data Visualization

- Designed a responsive dark-mode Security Operations Center (SOC) dashboard (code/templates/dashboard.html, code/static/style.css) with zero external CDN dependencies.
- Built custom inline SVG vector visualizations: a real-time radial continuous trust gauge and a 40-window rolling score history line chart with visual threshold zones.
- Implemented live biometric telemetry metric cards and an accelerated Replay Mode controller streaming pre-recorded sessions at 1 window/sec for frictionless evaluation.

Core Deliverables & Verified Artifacts:	code/main.py, code/auth.py, code/engine_adapter.py, code/static/dashboard.js, code/static/style.css, code/templates/dashboard.html, code/templates/login.html.
---	--

CLIENT-SERVER INTEGRATION & CONTINUOUS STREAMING ARCHITECTURE

Client-Side Layer: Monitored textarea • keydown/keyup handlers • performance.now() timing • In-browser HMAC-SHA256 via Web Crypto • Web Locks single-tab guard.
API Gateway & Transport: 10-second batch dispatch via POST /api/window • X-CSRF-Token authentication • Thread-safe RLock() session store • Max 120 window rolling buffer.
Evaluation Pipeline: engine_adapter.py • Feature extractor • Scikit-Learn Isolation Forest pipeline • Continuous trust mapper • Policy dispatch (ALLOW / PROMPT_REAUTH / LOCK_SESSION).

Ridhi Batra • Roll Number: 1024030464

Engineering Focus: Automated Testing Suite, Privacy Architecture Auditing & Formal IEEE/UML Documentation Lead

Week 1: Threat Modeling, Zero-Trust Access Analysis & Course Proposal

- Formulated the post-authentication threat landscape, analyzing workstation abandonment, token exfiltration, and physiological imposter differences.
- Defined quantitative Capstone engineering benchmarks: detection latency ($\leq 5.0s$), false positive rate ($\leq 5.0\%$), inactive leniency, and API overhead.
- Authored and compiled the formal UCS503P project proposal document in LaTeX (project-proposal/main.tex), securing approval from Prof. Asif.

Week 2: Privacy Architecture Auditing & Data Pipeline Hardening

- Conducted strict security and privacy audits under GDPR and India DPDP Act 2023 principles, verifying mathematical non-invertibility of client-side key hashes.
- Hardened command-line collector scripts against directory traversal attacks by implementing strict alphanumeric regex validation on user and session parameters.
- Drafted standardized operational protocols and recording checklists (docs/recording-and-demo-checklist.md) for repeatable, privacy-compliant data collection.

Week 3: Automated Integration Test Suite Engineering & Boundary Verification

- Engineered comprehensive test suites in code/tests/ (test_collector.py and test_dashboard.py) using Python unittest and Flask mock clients.
- Implemented headless listener mocking and boundary verifications for CSRF rejection, session isolation, malformed JSON handling, and inactive window preservation.
- Achieved 100% test pass rate across all 11 unit and integration test cases in 0.63 seconds, validating total system stability and error-handling paths.

Week 4: Formal Software Engineering Diagrams, Performance Benchmarking & Prototype Report

- Developed publication-grade software engineering models: DFD Levels 0–2, UML Use Case diagram, Relational ER schema, and UML Swimlane Activity diagram.
- Engineered the Master Gantt schedule (docs/ACA_Gantt_Chart.xlsx and PDF) across 23 tasks and 5 phases with dynamic progress formulas.
- Benchmarked prototype runtime metrics (0.42–0.88 ms inference, 12.3–24.1 ms API roundtrip, 38.4 MB RAM) and co-authored the Prototype Evaluation Report in LaTeX.

Core Deliverables & Verified Artifacts:	code/tests/test_collector.py, code/tests/test_dashboard.py, docs/Diagrams/* (DFD 0-2, UML Use Case, ER, Swimlane), docs/ACA_Gantt_Chart.*, project-report-prototype-stage/.*.
---	---

SUPERVISOR EVALUATION & REVIEW SIGN-OFF

Faculty Supervisor: Prof. Asif (Dr. Jeelani Asif)
Department: Computer Science & Engineering Department (CSED)
Institution: Thapar Institute of Engineering & Technology, Patiala
Evaluation Stage: Prototype Working Model Review (Mid-Semester)
Remarks: ☒ Satisfactory ☒ Good ☒ Excellent
Signature: _____ Date: _____

SOFTWARE ENGINEERING METRICS VERIFICATION

Automated Unit & Integration Tests: 11 / 11 Passed (100% Pass Rate)
Model Inference Latency: 0.42 ms – 0.88 ms (Target: ≤ 5.0 ms)
Client-to-API Roundtrip Latency: 12.3 ms – 24.1 ms (Target: ≤ 50.0 ms)
Privacy Hardening: Zero Plaintext Storage • Ephemeral HMAC-SHA256
Artifacts: DFD L0-L2 • UML Use Case • ER Schema • Master Gantt
Documentation: MkDocs Material • Academic LaTeX Reports